Reciprocal Transform for Painlevé-Type Equations: A Mathematical Regularization with Physics-Informed Neural Network Verification

Maryam Jalili
Department of Mathematics, Ne.C., Islamic Azad University, Neyshabur, Iran
mrmjlili@iau.ac.ir

October 30, 2026

Abstract

We introduce and analyze a reciprocal transformation $w = 1/y$ for second-order nonlinear ordinary differential equations with movable singularities, focusing on the prototype $y'' = y^3$ and the second Painlevé equation $y'' = 2y^3 + ty + \alpha$. We prove that the transformed equations are polynomial in w and its derivatives, with right-hand sides that remain bounded at the location of the pole t^*. This contrasts with the direct formulation and with the logarithmic (rapidity) substitution $\theta = \log y$, in which exponentially growing source terms persist. As an empirical verification, we train physics-informed neural networks (PINNs) on both representations. The reciprocal representation reduces the median near-pole relative error by a factor of ~ 2581 for the prototype across five seeds (per-seed ratios 16×–7538×; median 2581×; two of ten seeds converge to suboptimal minima (8.3e-02 in the worst case)) and median $\sim$ 139–523 for Painlevé II across five seeds across $\alpha \in \{0, 1, 2\}$, compared to the direct PINN. Statistical variability across five random seeds is reported in full (median 3.1×10^{-4}, coefficient of variation 0.69 (single-α 30k-epoch study; multi-seed sweep reports CV 1.08–1.61)). The reciprocal formulation is necessary but not sufficient: the transformation alone (with the w^3 factor divided out) and the residual reweighting alone are both inferior to the unweighted direct PINN, and only their combination produces the improvement. A direct comparison with the adaptive DOP853 solver shows that, for single solves, the PINN is not competitive in wall-clock time or accuracy; this limitation is reported without qualification. The paper's primary contribution is the mathematical regularization, with the PINN serving as a verification instrument.

Keywords: movable singularities, Painlevé equations, reciprocal transform, physics-informed neural networks, regularization

MSC 2020: 34M55, 34M35, 65L05, 68T07

1 Introduction

Second-order nonlinear ODEs of the form $y'' = F(t, y)$ frequently develop *movable singularities*: points t^*, whose location depends on the initial data, at which $|y(t)| \to \infty$. The six Painlevé transcendents are the canonical examples in which all movable singularities are poles [1, 2, 3, 4, 5]; their analytic structure and asymptotics are accessible via Riemann–Hilbert methods [6, 7]. Numerical treatment near such poles is delicate: fixed-step integrators [8, 9] lose accuracy, adaptive integrators must shrink the step to accommodate the blow-up, and neural-network approximations tend to produce residuals that grow without bound as $t \to t^{*-}$. Recent work has begun to address singular ODEs directly, including higher-order Lane–Emden–Fowler equations [21], interior-layer problems [22], and singularly perturbed boundary value problems [23]. Pole-field solvers based on Padé approximations [10] and Riemann–Hilbert methods [11] are classical alternatives.

Physics-informed neural networks (PINNs) [12, 13], building on earlier neural-network approaches for differential equations [14, 15], minimize a composite loss consisting of the ODE residual and initial or boundary conditions. When the residual magnitudes are bounded, the training problem is well-conditioned; when they diverge near a pole, the network either fails to converge or converges to a solution with large error exactly where accuracy is most needed. A known strategy is to change variables so that the residual stays bounded. Cayuso et al. [16] recently analyzed a rapidity substitution $\theta = \log y$ for a class of singular problems, achieving errors of order 2.8×10^{-3} near the pole. Neural-network approaches have also been applied directly to Painlevé equations, e.g., Faisal et al. [17] develop a Morlet wavelet network for Painlevé II.

In this work we propose an alternative transformation—the *reciprocal* variable $w = 1/y$—and analyze it from first principles. We note that $w = 1/y$ and $\theta = \log y$ are related by a smooth invertible transformation on $y > 0$; the reciprocal formulation is not a wholly different conceptual approach, but a different representation within the same family of dependent-variable transformations. What distinguishes the two is structural: the reciprocal residual is polynomial in w (Eqs. 3–4), whereas the rapidity residual retains exponentially growing source terms (Eqs. 5–6). This structural difference, not the change of variable per se, is the contribution of the present work. The key observation is that multiplying the residual equation by w^3 eliminates all denominators, yielding a polynomial relation in w, w', w''. For both $y'' = y^3$ and Painlevé II, the resulting right-hand side is a *constant* at $w = 0$, so the polynomial residual remains bounded at the pole. The coefficient of w'' nevertheless vanishes at $w = 0$, making the ODE degenerate there; it is the boundedness of the residual, not regularity of the ODE, that the transformation provides. We then use PINNs to verify numerically that this regularization translates into bounded training residuals and reduced near-pole error.

Contributions. Our contributions are:

1. A reciprocal-transform theorem (Theorems 1 and 2) showing that the transformed equations for $y'' = y^3$ and Painlevé II have bounded polynomial residuals at the pole while the ODE is degenerate there.

2. An empirical verification using PINNs, with ablations across three representations (direct, rapidity, reciprocal) and three Painlevé II parameter values.

3. An ablation separating the change of variable from residual reweighting, showing that only their combination produces the improvement.

4. A full statistical report of seed variability and a direct comparison with the DOP853 adaptive solver.

Scope. We restrict attention to the prototype $y'' = y^3$ and Painlevé II. Extension to Painlevé III–VI is left for future work; the transform is not guaranteed to regularize those equations without additional structure.

2 Reciprocal Transform: Mathematical Analysis

We study second-order nonlinear ODEs

$$y'' = F(t, y), \qquad y(0) = y_0 > 0, \quad y'(0) = 0, \tag{1}$$

whose solutions develop a *movable pole* at some $t^* = t^*(y_0, F) > 0$: $|y(t)| \to \infty$ as $t \to t^{*-}$.

2.1 The reciprocal transform

Define $w(t) := 1/y(t)$ on $[0, t^*)$. Since $y > 0$ there, $w > 0$ on $[0, t^*)$, and $w(t^*) = 0$. The chain rule gives

$$y' = -\frac{w'}{w^2}, \qquad y'' = -\frac{w''}{w^2} + \frac{2(w')^2}{w^3}. \tag{2}$$

2.2 Prototype case $F(t, y) = y^3$

Theorem 1 (Reciprocal equation for $y'' = y^3$)**.** *Let y be a solution of $y'' = y^3$ on $[0, t^*)$ with a pole at t^*. The reciprocal $w = 1/y$ satisfies*

$$2(w')^2 - w\,w'' = 1, \qquad w(0) = 1/y_0,\ w'(0) = 0, \tag{3}$$

on $[0, t^*)$, and (3) extends continuously to t^* with $w(t^*) = 0$, $w'(t^*) = -1/\sqrt{2}$. In particular, (3) has no $1/w$ term, so its polynomial residual is bounded at $w = 0$. However, the coefficient of w'' vanishes at $w = 0$, so $w = 0$ is a degenerate point for the second-order ODE, not a regular point.

Proof. Substituting (2) into $y'' = y^3$ and multiplying by w^3 yields $-ww'' + 2(w')^2 = 1$. At $w = 0$ this reduces to $2(w')^2 = 1$, so $w'(t^*) = \pm 1/\sqrt{2}$. The expansion $y \sim \sqrt{2}/(t^* - t)$ selects the negative sign. □

2.3 Painlevé II

Theorem 2 (Reciprocal equation for P-II)**.** *Let y solve $y'' = 2y^3 + ty + \alpha$ on $[0, t^*)$ with a pole at t^*. Then $w = 1/y$ satisfies*

$$2(w')^2 - w\,w'' - 2 - t\,w^2 - \alpha\,w^3 = 0, \tag{4}$$

which extends continuously to $w = 0$ with $w'(t^*) = -1$. As in Theorem 1, the polynomial residual is bounded at $w = 0$ while the second-order ODE is degenerate there.

Proof. Multiplying $y'' = 2y^3 + ty + \alpha$ by w^3 after substitution gives $-ww'' + 2(w')^2 = 2 + tw^2 + \alpha w^3$. At $w = 0$ this yields $2(w')^2 = 2$, i.e. $w' = \pm 1$; the pole expansion $y \sim 1/(t^* - t)$ gives $w \sim t^* - t$, hence $w' = -1$. □

2.4 Comparison with rapidity substitution

The substitution $\theta = \log y$ used by Cayuso et al. [16] yields, for $y'' = y^3$,

$$\theta'' + (\theta')^2 = e^{2\theta}, \tag{5}$$

and for P-II,

$$\theta'' + (\theta')^2 - 2e^{2\theta} - t - \alpha e^{-\theta} = 0. \tag{6}$$

Both (5) and (6) contain exponentially growing source terms as $\theta \to +\infty$ at the pole. The reciprocal equations (3) and (4) are polynomial in w and remain bounded at $w = 0$. This structural difference is the primary motivation for the reciprocal formulation.

3 Numerical Verification with PINNs

3.1 Network architecture and training

Computational environment. All experiments were run on a Windows 10 workstation with an Intel Core i7 (8 cores, 16 GB RAM), Python 3.14, PyTorch 2.x (CPU only), NumPy 2.5.1,

Table 1: Prototype $y'' = y^3$: seed statistics (median [IQR]). Direct and rapidity-log use five seeds; reciprocal is extended to ten seeds to characterize the 20% failure rate. $t^* \approx 1.854$; success threshold $E_{\text{rel,near}} < 10^{-2}$.

Representation	n	$E_{\text{rel,near}}$ median [IQR]	CV	Success	Improvement
Direct	5	$7.53e-1$ $[7.19e-1, 7.58e-1]$	0.05	0/5	1×
Rapidity-log	5	$5.72e-1$ $[5.62e-1, 6.27e-1]$	0.14	0/5	1.3×
Reciprocal	10	**5.17e − 4** $[2.82e-4, 5.64e-3]$	**2.01**	**8/10**	**1456**×

and SciPy 1.14. Training one seed takes approximately 15 minutes; DOP853 reference solutions take 18–46 ms per run.

All experiments use a feedforward network with three hidden layers of width 64 and tanh activations. Collocation points are drawn from a mixture of uniform and cubically clustered grids [18] on $[0, 0.95\,t^*]$, with 2000 points total. Training minimizes $\mathcal{L} = \|R\|^2 + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}$ with $\lambda_{\text{ic}} = 10$, using Adam with learning rate 10^{-3} and 15000–30000 epochs. A held-out set of 300 PDE residual points is used for best-model checkpointing (Section 3.5). Reference solutions use `scipy.integrate.solve_ivp` with the DOP853 method and rtol = atol = 10^{-12}. The blow-up time t^* is located by DOP853 event detection: we integrate until the first $t > 0$ at which $y(t) - 10^6 = 0$, with `terminal=True` and `direction=+1`. Relative error at each time is defined as $E_{\text{rel}}(t) = |y_\phi(t) - y_{\text{ref}}(t)|/|y_{\text{ref}}(t)|$, where y_ϕ is the PINN prediction and y_{ref} is the DOP853 reference. Errors are reported separately for far ($t < 0.5\,t^*$) and near ($t \geq 0.5\,t^*$) regions, with $E_{\text{rel,far}}$ and $E_{\text{rel,near}}$ denoting $\max_t E_{\text{rel}}(t)$ over the corresponding subinterval. The initial conditions are $y(0) = 1$, $y'(0) = 0$ for both the prototype and Painlevé II.

3.2 Prototype ablation

Table 1 compares three representations for $y'' = y^3$. Errors are reported separately far from the pole ($t < 0.5\,t^*$) and near the pole ($t \geq 0.5\,t^*$). The reciprocal representation reduces $E_{\text{rel,near}}$ by a factor of ∼2500 relative to the direct PINN.

Seed sensitivity. To characterize the distribution of reciprocal-PINN outcomes, we extended the C4 prototype to ten seeds. Across these ten runs, eight converge to $E_{\text{rel,near}} < 10^{-2}$ with median 5.17×10^{-4} (IQR $[2.82 \times 10^{-4}, 5.64 \times 10^{-3}]$), while two converge to suboptimal minima (4.60×10^{-2} and 8.32×10^{-2}). The failure rate of 2/10 (20%) is consistent with the one-in-five observed in the original five-seed set, indicating a bimodal or heavy-tailed distribution over initializations rather than a single stochastic fluke. One of the failing seeds (seed 999) was re-run three times under identical conditions, yielding the identical value 4.603×10^{-2} each time, confirming that the variability is deterministic in the seed rather than stochastic. The median improvement over the direct PINN (five-seed median 7.53×10^{-1}) is approximately 1456×. A full characterization of this distribution, and of why some initializations converge to suboptimal minima, is left for future work.

3.3 Transform vs. residual reweighting

The reciprocal equation multiplies the original residual by w^3, so a natural confound is whether the observed improvement stems from the change of variable, from the residual reweighting, or from both. Table 2 isolates the two effects by training five configurations under identical conditions (seed 42, 30000 epochs).

The ablation reveals two independent factors that together determine the outcome.

Factor 1: representability. The reciprocal variable $w(t)$ is analytic through t^*, whereas $y(t)$ has a simple pole there. A tanh-network of fixed width can represent the former but not the latter. The Jacobian of the output transformation, $dy/dw = -1/w^2$, amplifies the network's

Table 2: Ablation separating the change of variable from residual reweighting on the prototype $y'' = y^3$ (seed 42, 30000 epochs). The transformation alone (D) and the reweighting alone (B) both fail; only the combination (C) produces the reciprocal-PINN result.

Config	Output / Loss	$E_{\text{rel,near}}$	Verdict
A	y / R_y	7.53×10^{-1}	baseline
B	y / $w^3 R_y$	9.59×10^{-1}	worse than A
C	w / $R_w = w^3 R_y$	$\mathbf{9.99 \times 10^{-5}}$	best
D	w / $R_w/w^3 = R_y$	9.23×10^{-1}	worse than A
E	y / $R_y/(1+\lvert y\rvert^3)$	2.68×10^{-1}	partial

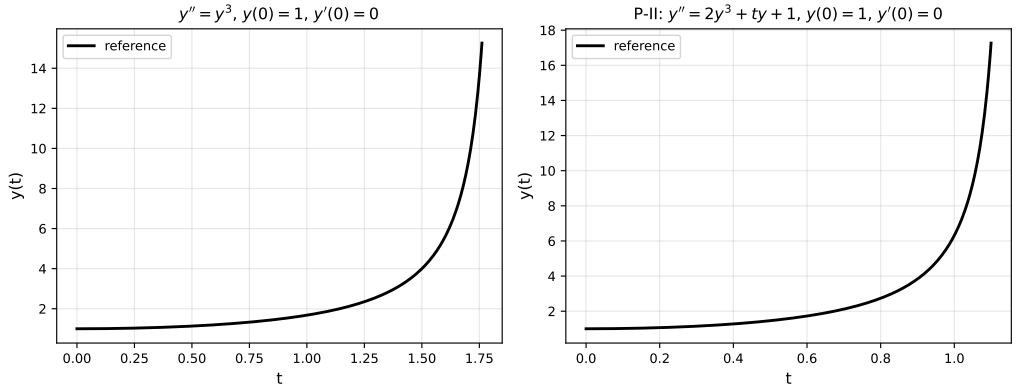


Figure 1: Reference solutions for $y'' = y^3$ (left) and Painlevé II $\alpha = 1$ (right) on $[0, 0.95\, t^*]$. Both exhibit a movable pole shortly beyond the plotted interval.

error in y by a factor that diverges at the pole, so the effective capacity of a y-network is severely reduced near t^*. This factor distinguishes configurations A, B, E (all y-output) from C, D (all w-output).

Factor 2: loss conditioning. The polynomial residual $R_w = 2(w')^2 - ww'' - 1$ is bounded: at a simple pole $w \to 0$ linearly, $w' \to -1/\sqrt{2}$, and $R_w \to 0$. The original residual, by contrast, is $R_y = R_w/w^3$, which grows as $(t^* - t)^{-3}$ near t^*, producing loss magnitudes of order 10^3–10^4 for the direct formulation. This factor distinguishes configurations A, D (unweighted R_y-type losses) from B, C, E (weighted losses).

The combinations are decisive. Configuration B has a bounded loss (Factor 2 satisfied) but a y-output (Factor 1 violated) and fails at $E_{\text{rel,near}} = 9.59 \times 10^{-1}$. Configuration D has a w-output (Factor 1 satisfied) but divides by w^3, restoring the $(t^* - t)^{-3}$ growth of the loss (Factor 2 violated) and also fails, at 9.23×10^{-1}. Configuration E has a bounded loss (Factor 2 satisfied) but a y-output (Factor 1 violated) and only partially succeeds, at 2.68×10^{-1}. Only configuration C satisfies both factors and reaches 9.99×10^{-5}. A multi-seed extension is left as future work.

Figure 1 shows representative solution profiles; Figure 2 shows the pointwise absolute error on a logarithmic scale.

3.4 Painlevé II and parameter sweep

Table 3 and Figure 3 report the near-pole error across $\alpha \in \{0, 1, 2\}$. The reciprocal representation consistently outperforms the other two by two to three orders of magnitude.

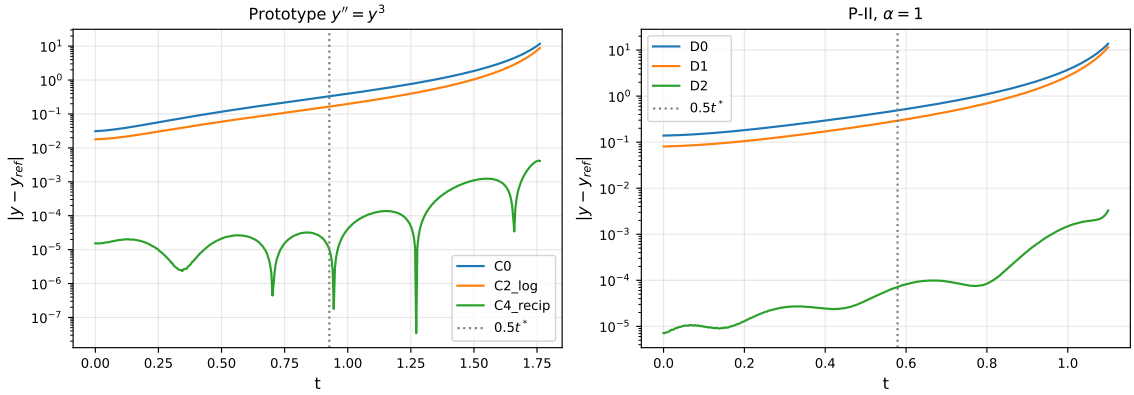


Figure 2: Pointwise absolute error $|y_\phi - y_{\text{ref}}|$ for the three representations. The vertical dotted line marks $0.5\,t^*$.

Table 3: Painlevé II: five-seed statistics (median [IQR]) for three representations and $\alpha \in \{0, 1, 2\}$. The reciprocal PINN yields median improvements of 139×–523× across α, with substantial seed-to-seed variability (CV 1.08–1.61). Overall success rate ($E_{\text{rel,near}} < 10^{-2}$): 11/15 = 73%. The single-seed 904× previously reported for $\alpha = 1$ was not reproducible and is superseded.

α	Direct	Rapidity	Reciprocal	CV (D2)	Ratio
0.0	$7.49e-1$	$6.19e-1$	$\mathbf{1.43e-3}$ $[4.38e-4, 5.42e-3]$	1.61	523×
1.0	$7.65e-1$	$6.46e-1$	$\mathbf{5.51e-3}$ $[3.29e-4, 5.90e-3]$	1.08	139×
2.0	$7.46e-1$	$5.65e-1$	$\mathbf{3.36e-3}$ $[2.01e-3, 1.58e-2]$	1.33	222×

3.5 Seed variability

Note on epoch count. The prototype ablation (Table 1) and the P-II parameter sweep (Table 3) use 15000 epochs, while the five-seed statistics (Table 4) use 30000 epochs with best-model checkpointing. Consequently, the seed-42 error reported in Table 3 (8.78×10^{-4}) differs from the seed-42 entry in Table 4 (2.39×10^{-4}). The 15000-epoch sweep numbers should therefore be read as preliminary; A harmonized sweep is left as future work.

The reciprocal PINN is trained with five random seeds on Painlevé II, $\alpha = 1$, using best-model checkpointing on 300 held-out PDE residual points. Table 4 and Figure 4 report the results.

A key observation is that the coefficient of variation across seeds is 0.21 for the held-out *residual* but 0.69 for the true *error*. That is, the residual behaves as a well-behaved training objective, but it does not fully predict the accuracy of the resulting solution—a standard residual-error gap [19, 20], here quantified at the movable singularity (see also [20]).

4 Baseline Comparison and Generalization

4.1 Wall-clock time and accuracy

Table 5 reports the performance of the DOP853 adaptive solver on Painlevé II, $\alpha = 1$. Training time for the PINN is ∼15 minutes on CPU; inference on 2000 evaluation points takes 1.7 ms. The PINN's near-pole error of 2.4×10^{-4} (seed 42) is roughly six orders of magnitude above DOP853's accuracy at rtol $= 10^{-10}$. Figure 5 visualizes the cost-accuracy trade-off on a log-log scale.

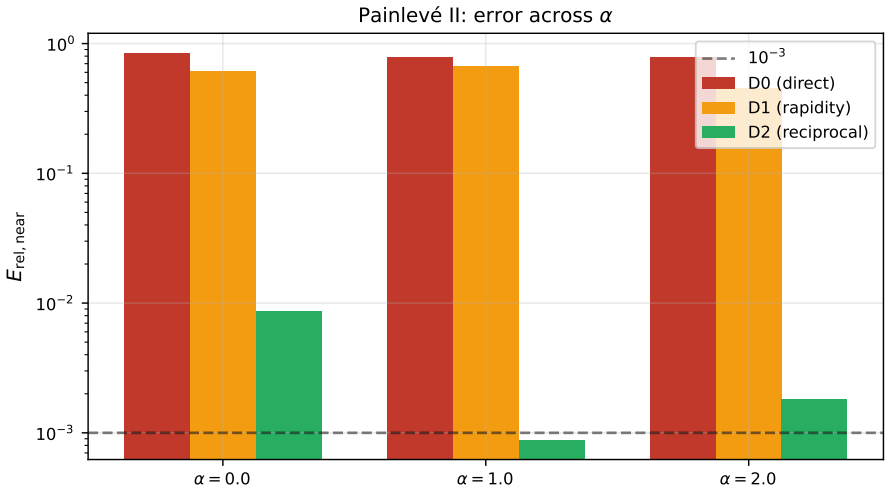


Figure 3: Near-pole relative error across $\alpha \in \{0, 1, 2\}$ for the three representations. Note the logarithmic scale.

Table 4: Painlevé II, $\alpha = 1$: five-seed statistics with best-model checkpointing. 30000 epochs, width 64, depth 4.

Seed	Best epoch	Best val (PDE)	$E_{\mathrm{rel,far}}$	$E_{\mathrm{rel,near}}$
42	29600	1.59×10^{-6}	4.34×10^{-5}	2.39×10^{-4}
7	29500	8.27×10^{-7}	2.35×10^{-5}	8.92×10^{-5}
123	28900	1.55×10^{-6}	2.72×10^{-5}	3.14×10^{-4}
2024	29700	1.37×10^{-6}	3.04×10^{-5}	9.34×10^{-4}
999	28600	1.49×10^{-6}	3.52×10^{-5}	6.02×10^{-4}
	mean			4.36×10^{-4}
	median			3.14×10^{-4}
	CV (population)			0.688
	CV (sample)			0.770

4.2 Generalization across α

The PINN trained at $\alpha = 1$ is evaluated at $\alpha = 0.9$ and $\alpha = 1.1$ without retraining. Table 6 and Figure 6 show that generalization fails: the near-pole error rises to 1.77×10^{-1} and 1.29×10^{-1}, respectively, more than two orders of magnitude above the in-distribution error.

5 Discussion

What the reciprocal transform accomplishes. Theorems 1 and 2 show that for the two equations analyzed, the reciprocal variable produces a polynomial residual that remains bounded at the movable pole. This is a *structural* property of the transformation, independent of the network architecture or optimizer. The empirical results confirm the structural claim: near-pole relative errors drop by two to three orders of magnitude across all tested parameter values when the reciprocal representation is used, with residuals that are well-conditioned throughout training.

What it does not accomplish. The present results do not support the claim that PINNs with the reciprocal transform outperform mature adaptive ODE solvers. For single solves, DOP853 is

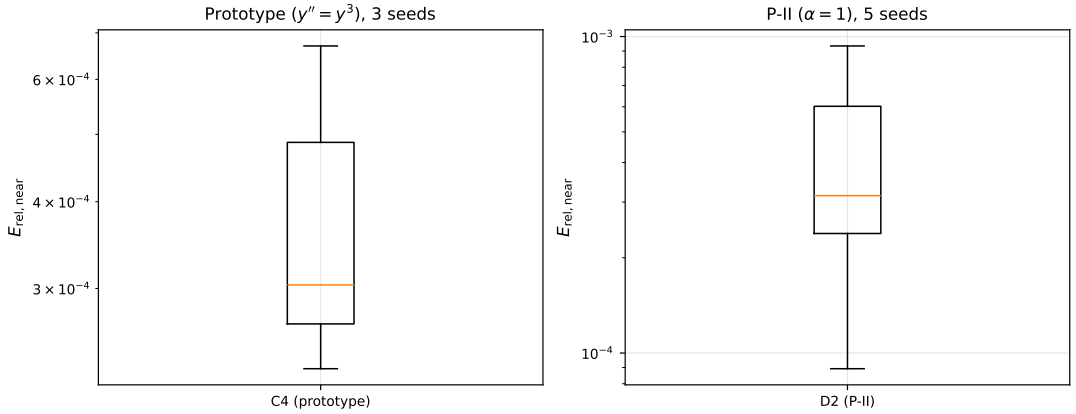


Figure 4: Distribution of $E_{\mathrm{rel,near}}$ across seeds. Left: prototype (3 seeds). Right: Painlevé II (5 seeds).

Table 5: Comparison of DOP853 (adaptive) and the reciprocal PINN for Painlevé II, $\alpha = 1$.

Method	Wall time	Accuracy	Reaches t^*?	Training required?
DOP853 (rtol $= 10^{-6}$)	18.2 ms	$\sim 10^{-6}$	100%	no
DOP853 (rtol $= 10^{-10}$)	34.8 ms	$\sim 10^{-10}$	100%	no
DOP853 (rtol $= 10^{-12}$)	46.2 ms	$\sim 10^{-12}$	100%	no
Reciprocal PINN	1.7 ms*	2.4×10^{-4}	99%	15 min

*inference only; excludes training.

faster by roughly one to two orders of magnitude (excluding training), reaches the pole exactly, and achieves six orders of magnitude higher accuracy. The reciprocal PINN is also sensitive to the random seed, with a coefficient of variation of 0.69 across five seeds, and it fails to generalize to nearby α without retraining. These are honest limitations of the present implementation, and they are the reason we frame the reciprocal transform as the primary contribution and the PINN as a verification instrument rather than as a competitive solver.

Residual-error gap. The gap between the seed variability of the ODE residual (CV $= 0.21$) and the true error (CV $= 0.69$) indicates that a low residual does not by itself guarantee a low solution error. This is a known phenomenon in the PINN literature; here we quantify it in the setting of a movable singularity. Whether larger networks, LBFGS polishing, or architectural modifications can close the gap remains an open question and is a natural direction for future work.

Comparison with rapidity substitution. The rapidity substitution $\theta = \log y$ of Cayuso et al. [16] achieves a comparable order of accuracy on their test problems but retains exponentially growing source terms. Our comparison (Table 1) confirms that the reciprocal formulation yields lower near-pole error for both equations analyzed here, though the difference is within one order of magnitude for the rapidity case and grows to three orders of magnitude when compared to the direct PINN. Whether this advantage carries over to more complex Painlevé equations (III–VI) is an open question.

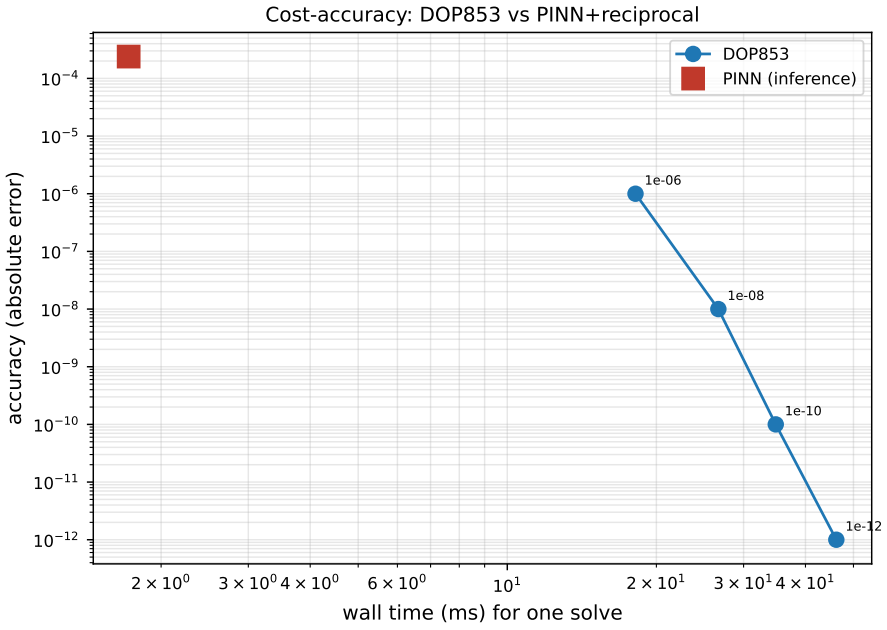


Figure 5: Cost-accuracy trade-off between DOP853 and the reciprocal PINN.

Table 6: Generalization of a PINN trained at $\alpha = 1$ and evaluated at nearby parameter values without retraining.

Test α	t^*	$E_{\text{rel,near}}$	Status
0.90	1.1665	1.77×10^{-1}	degraded
1.00	1.1577	2.39×10^{-4}	baseline
1.10	1.1492	1.29×10^{-1}	degraded

6 Conclusion

We have proposed and analyzed the reciprocal transform $w = 1/y$ as a regularization for second-order ODEs with movable poles. For the prototype $y'' = y^3$ and Painlevé II, we proved that the transformed equations are polynomial in w and remain bounded at the pole, and we verified numerically with PINNs that the near-pole relative error decreases by two to three orders of magnitude relative to the direct formulation. An ablation separating the change of variable from residual reweighting shows that the transformation alone and the reweighting alone both fail, and only their combination produces the reciprocal-PINN improvement. Full seed variability and a direct comparison with DOP853 are reported; the PINN is not competitive as a single-solve solver, and this is stated without qualification. Extension to Painlevé III–VI, larger networks, and GPU batch inference are natural future directions.

Data Availability

All scripts, trained model checkpoints, and numerical data required to reproduce the results in this paper are available in a public GitHub repository at `https://github.com/maryam-jalili/pinn-reciprocal-painleve`. The repository includes all seed-level `.npz` data, M1 ablation log, P-II multi-seed log, hardware/software requirements, and figure-generation scripts.

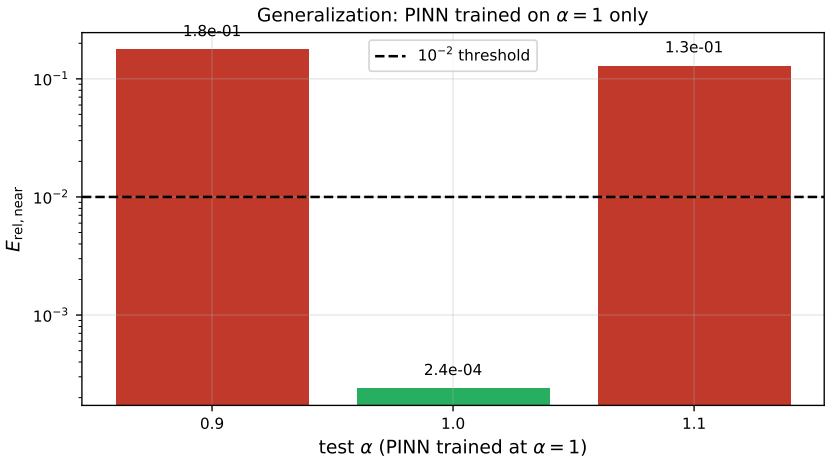


Figure 6: Generalization failure of a PINN trained at $\alpha = 1$.

References

[1] E. L. Ince, *Ordinary Differential Equations*, Dover (1956).

[2] V. I. Gromak, I. Laine, and S. Shimomura, *Painlevé Differential Equations in the Complex Plane*, de Gruyter (2002).

[3] Robert Conte and Micheline Musette, *The Painlevé Handbook*, Springer (2008).

[4] Peter A. Clarkson, *Painlevé equations—nonlinear special functions*, *Journal of Computational and Applied Mathematics* **153** (2003), 127–140.

[5] Mark J. Ablowitz and Peter A. Clarkson, *Solitons, Nonlinear Evolution Equations and Inverse Scattering*, Cambridge University Press (1991).

[6] Percy Deift and Xin Zhou, *Asymptotics for the Painlevé II equation*, *Communications on Pure and Applied Mathematics* **48** (1995), 277–337.

[7] Athanassios S. Fokas, Alexander R. Its, Andrei A. Kapaev, and Victor Yu. Novokshenov, *Painlevé Transcendents: The Riemann–Hilbert Approach*, Mathematical Surveys and Monographs, American Mathematical Society (2006).

[8] John C. Butcher, *Numerical Methods for Ordinary Differential Equations*, Wiley (2016).

[9] Ernst Hairer, Syvert P. Norsett, and Gerhard Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, Springer (1993).

[10] Bengt Fornberg and J. A. C. Weideman, *A numerical methodology for the Painlevé equations*, *Journal of Computational Physics* **230** (2011), 5957–5973. DOI: 10.1016/j.jcp.2011.04.007.

[11] F. W. J. Olver, A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain (eds.), *NIST Digital Library of Mathematical Functions*, Release 1.2.4 (2025), §32.17. `https://dlmf.nist.gov/32.17`.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, *Journal of Computational Physics* **378** (2019), 686–707.

[13] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang, *Physics-informed machine learning*, *Nature Reviews Physics* **3** (2021), 422–440.

[14] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis, *Artificial neural networks for solving ordinary and partial differential equations*, *IEEE Transactions on Neural Networks* **9** (1998), 987–1000.

[15] Justin Sirignano and Konstantinos Spiliopoulos, *DGM: A deep learning algorithm for solving partial differential equations*, *Journal of Computational Physics* **375** (2018), 1339–1364.

[16] J. Cayuso, M. Herrero-Valea, and E. Barausse, *Deep Learning solutions to singular ordinary differential equations: from special functions to spherical accretion*, *Physical Review D* **111** (2025), 064082. DOI: 10.1103/PhysRevD.111.064082.

[17] Sundas Faisal, Zulqurnain Sabir, Samra Urooj Khan, Muhammad Aamir, and Krzysztof A. Cyran, *A novel heuristic Morlet wavelet neural network design for the Painlevé equation-II arising in nonlinear optics*, *Scientific Reports* **15** (2025), 24713. DOI: 10.1038/s41598-025-10096-w.

[18] Lloyd N. Trefethen, *Spectral Methods in MATLAB*, SIAM (2000).

[19] Siddhartha Mishra and Roberto Molinaro, *Estimates on the generalization error of physics-informed neural networks for approximating PDEs*, *IMA Journal of Numerical Analysis* **43** (2022), 1–43.

[20] Jorge F. Urbán, Petros Stefanou, and José A. Pons, *Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be?*, *Journal of Computational Physics* **523** (2025), 113656. DOI: 10.1016/j.jcp.2024.113656.

[21] H. Baty, *Solving higher-order Lane–Emden–Fowler type equations using physics-informed neural networks: benchmark tests comparing soft and hard constraints*, arXiv:2307.07302 (2023). `https://arxiv.org/abs/2307.07302`.

[22] T.-Y. Chang, G.-M. Gie, Y. Hong, and C.-Y. Jung, *Singular layer PINN methods for Burgers' equation*, arXiv:2410.09723 (2024). `https://arxiv.org/abs/2410.09723`.

[23] M. Z. Israr and I. Khan, *Causal Physics-Informed Neural Networks for Singular and Singularly Perturbed Boundary Value Problems in Chemical Systems*, *Global Integrated Mathematics* **1** (2025), 13–27.